DSA Patterns Guide

15 Core Coding Patterns — From Basics to Interview-Ready

Python **Java** **C++** **JavaScript**

Har pattern mein: kab use karna hai, time/space complexity, ek example problem, aur uska working code — 4 languages mein.

1. Two Pointers
2. Sliding Window
3. Fast & Slow Pointers
4. Merge Intervals
5. Cyclic Sort
6. In-place Reversal of a Linked List
7. Tree BFS (Level Order Traversal)
8. Tree DFS
9. Subsets / Backtracking
10. Modified Binary Search
11. Top K Elements (Heap)
12. Dynamic Programming (0/1 Knapsack)
13. Graph BFS / DFS
14. Monotonic Stack
15. Union-Find (Disjoint Set)

1. Two Pointers

Kab use karein	Array or string is sorted (or can be sorted), and you need to find pairs/triplets satisfying a condition, or compare elements from both ends.
Complexity	Usually O(n) time, O(1) extra space — turns a brute-force O(n²) into a single pass.
Example problem	Pair with Target Sum: Given a sorted array, find two numbers that add up to a target value.

Python

```
def pair_with_target_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current_sum = arr[left] + arr[right]
        if current_sum == target:
            return [left, right]
        if current_sum < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]

print(pair_with_target_sum([1, 2, 3, 4, 6], 6))  # [1, 3]
```

Java

```
public int[] pairWithTargetSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int currentSum = arr[left] + arr[right];
        if (currentSum == target) {
            return new int[] { left, right };
        }
        if (currentSum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[] { -1, -1 };
}
```

C++

```
#include <vector>
using namespace std;

vector<int> pairWithTargetSum(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left < right) {
        int currentSum = arr[left] + arr[right];
        if (currentSum == target) {
            return {left, right};
        }
        if (currentSum < target) {
            left++;
        } else {
            right--;
        }
    }
    return {-1, -1};
}
```

JavaScript

```
function pairWithTargetSum(arr, target) {
  let left = 0, right = arr.length - 1;
  while (left < right) {
    const currentSum = arr[left] + arr[right];
    if (currentSum === target) {
      return [left, right];
    }
    if (currentSum < target) {
      left++;
    } else {
      right--;
    }
  }
  return [-1, -1];
}
```

2. Sliding Window

Kab use karein	Problems asking for the best (max/min/longest/shortest) contiguous subarray or substring, especially with a fixed or variable-size window.
Complexity	O(n) time — each element is added to and removed from the window at most once, instead of recomputing sums for every window (O(n·k) brute force).
Example problem	Maximum Sum Subarray of Size K: Find the maximum sum of any contiguous subarray of size k.

Python

```python
def max_sum_subarray(arr, k):
    max_sum, window_sum = 0, 0
    window_start = 0
    for window_end in range(len(arr)):
        window_sum += arr[window_end]
        if window_end >= k - 1:
            max_sum = max(max_sum, window_sum)
            window_sum -= arr[window_start]
            window_start += 1
    return max_sum

print(max_sum_subarray([2, 1, 5, 1, 3, 2], 3))  # 9
```

Java

```java
public int maxSumSubarray(int[] arr, int k) {
    int maxSum = 0, windowSum = 0, windowStart = 0;
    for (int windowEnd = 0; windowEnd < arr.length; windowEnd++) {
        windowSum += arr[windowEnd];
        if (windowEnd >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= arr[windowStart];
            windowStart++;
        }
    }
    return maxSum;
}
```

C++

```cpp
#include <vector>
#include <algorithm>
using namespace std;

int maxSumSubarray(vector<int>& arr, int k) {
    int maxSum = 0, windowSum = 0, windowStart = 0;
    for (int windowEnd = 0; windowEnd < (int)arr.size(); windowEnd++) {
        windowSum += arr[windowEnd];
        if (windowEnd >= k - 1) {
            maxSum = max(maxSum, windowSum);
            windowSum -= arr[windowStart];
            windowStart++;
        }
    }
    return maxSum;
}
```

JavaScript

```javascript
function maxSumSubarray(arr, k) {
  let maxSum = 0, windowSum = 0, windowStart = 0;
  for (let windowEnd = 0; windowEnd < arr.length; windowEnd++) {
    windowSum += arr[windowEnd];
    if (windowEnd >= k - 1) {
      maxSum = Math.max(maxSum, windowSum);
      windowSum -= arr[windowStart];
      windowStart++;
    }
  }
  return maxSum;
}
```

3. Fast & Slow Pointers

Kab use karein	Linked list or array problems involving cycles, finding the middle element, or detecting a repeated sequence — without extra memory.
Complexity	O(n) time, O(1) space, based on Floyd's Cycle Detection algorithm.
Example problem	Linked List Cycle: Determine whether a linked list has a cycle.

Python

```
class ListNode:
    def __init__(self, value):
        self.value = value
        self.next = None

def has_cycle(head):
    slow, fast = head, head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Java

```
class ListNode {
    int value;
    ListNode next;
    ListNode(int value) { this.value = value; }
}

public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            return true;
        }
    }
    return false;
}
```

C++

```
struct ListNode {
    int value;
    ListNode* next;
    ListNode(int value) : value(value), next(nullptr) {}
};

bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            return true;
        }
    }
    return false;
}
```

JavaScript

```
class ListNode {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}

function hasCycle(head) {
  let slow = head, fast = head;
  while (fast !== null && fast.next !== null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) {
      return true;
    }
  }
  return false;
}
```

4. Merge Intervals

Kab use karein	Problems dealing with overlapping intervals — merging, inserting, or finding conflicts (meeting rooms, calendars, ranges).
Complexity	O(n log n) time due to sorting, O(n) space for the result.
Example problem	Merge Intervals: Merge all overlapping intervals into non-overlapping ones.

Python

```
def merge(intervals):
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        last_end = merged[-1][1]
        if start <= last_end:
            merged[-1][1] = max(last_end, end)
        else:
            merged.append([start, end])
    return merged

print(merge([[1, 3], [2, 6], [8, 10], [15, 18]]))  # [[1,6],[8,10],[15,18]]
```

Java

```
import java.util.*;

public int[][] merge(int[][] intervals) {
    if (intervals.length == 0) return new int[0][];
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    merged.add(intervals[0]);
    for (int i = 1; i < intervals.length; i++) {
        int[] last = merged.get(merged.size() - 1);
        if (intervals[i][0] <= last[1]) {
            last[1] = Math.max(last[1], intervals[i][1]);
        } else {
            merged.add(intervals[i]);
        }
    }
    return merged.toArray(new int[merged.size()][]);
}
```

C++

```cpp
#include <vector>
#include <algorithm>
using namespace std;

vector<vector<int>> merge(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> merged = {intervals[0]};
    for (size_t i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] <= merged.back()[1]) {
            merged.back()[1] = max(merged.back()[1], intervals[i][1]);
        } else {
            merged.push_back(intervals[i]);
        }
    }
    return merged;
}
```

JavaScript

```javascript
function merge(intervals) {
  if (intervals.length === 0) return [];
  intervals.sort((a, b) => a[0] - b[0]);
  const merged = [intervals[0]];
  for (let i = 1; i < intervals.length; i++) {
    const last = merged[merged.length - 1];
    if (intervals[i][0] <= last[1]) {
      last[1] = Math.max(last[1], intervals[i][1]);
    } else {
      merged.push(intervals[i]);
    }
  }
  return merged;
}
```

5. Cyclic Sort

Kab use karein	Array contains numbers in a known range like 1..n or 0..n, and you need to find missing, duplicate, or misplaced numbers in-place.
Complexity	O(n) time, O(1) extra space — each number is placed at its correct index in one pass.
Example problem	Find the Missing Number in an array containing n distinct numbers from 0 to n.

Python

```
def find_missing_number(nums):
    i, n = 0, len(nums)
    while i < n:
        j = nums[i]
        if j < n and nums[i] != nums[j]:
            nums[i], nums[j] = nums[j], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i:
            return i
    return n

print(find_missing_number([4, 0, 3, 1]))  # 2
```

Java

```
public int findMissingNumber(int[] nums) {
    int i = 0, n = nums.length;
    while (i < n) {
        int j = nums[i];
        if (j < n && nums[i] != nums[j]) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        } else {
            i++;
        }
    }
    for (i = 0; i < n; i++) {
        if (nums[i] != i) return i;
    }
    return n;
}
```

C++

```cpp
#include <vector>
using namespace std;

int findMissingNumber(vector<int>& nums) {
    int i = 0, n = nums.size();
    while (i < n) {
        int j = nums[i];
        if (j < n && nums[i] != nums[j]) {
            swap(nums[i], nums[j]);
        } else {
            i++;
        }
    }
    for (i = 0; i < n; i++) {
        if (nums[i] != i) return i;
    }
    return n;
}
```

JavaScript

```javascript
function findMissingNumber(nums) {
  let i = 0;
  const n = nums.length;
  while (i < n) {
    const j = nums[i];
    if (j < n && nums[i] !== nums[j]) {
      [nums[i], nums[j]] = [nums[j], nums[i]];
    } else {
      i++;
    }
  }
  for (i = 0; i < n; i++) {
    if (nums[i] !== i) return i;
  }
  return n;
}
```

6. In-place Reversal of a Linked List

Kab use karein	You need to reverse a linked list (fully or in k-sized groups) without using O(n) extra space.
Complexity	O(n) time, O(1) space.
Example problem	Reverse a Singly Linked List in place.

Python

```
class ListNode:
    def __init__(self, value):
        self.value = value
        self.next = None

def reverse(head):
    prev = None
    while head is not None:
        next_node = head.next
        head.next = prev
        prev = head
        head = next_node
    return prev
```

Java

```
class ListNode {
    int value;
    ListNode next;
    ListNode(int value) { this.value = value; }
}

public ListNode reverse(ListNode head) {
    ListNode prev = null;
    while (head != null) {
        ListNode next = head.next;
        head.next = prev;
        prev = head;
        head = next;
    }
    return prev;
}
```

C++

```
struct ListNode {
    int value;
    ListNode* next;
    ListNode(int value) : value(value), next(nullptr) {}
};

ListNode* reverse(ListNode* head) {
    ListNode* prev = nullptr;
    while (head != nullptr) {
        ListNode* next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}
```

JavaScript

```
class ListNode {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}

function reverse(head) {
  let prev = null;
  while (head !== null) {
    const next = head.next;
    head.next = prev;
    prev = head;
    head = next;
  }
  return prev;
}
```

7. Tree BFS (Level Order Traversal)

Kab use karein	You need to traverse a tree level by level — problems about tree depth, level averages, zigzag traversal, or the widest level.
Complexity	O(n) time, O(w) space where w is the maximum width of the tree.
Example problem	Binary Tree Level Order Traversal: Return node values grouped by level.

Python

```
from collections import deque

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def level_order(root):
    result = []
    if root is None:
        return result
    queue = deque([root])
    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

Java

```
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.add(node.left);
            if (node.right != null) queue.add(node.right);
        }
        result.add(level);
    }
    return result;
}
```

C++

```
#include <vector>
#include <queue>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int val) : val(val), left(nullptr), right(nullptr) {}
};

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> result;
    if (root == nullptr) return result;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int levelSize = q.size();
        vector<int> level;
        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        result.push_back(level);
    }
    return result;
}
```

JavaScript

```
class TreeNode {
  constructor(val) {
    this.val = val;
    this.left = null;
    this.right = null;
  }
}

function levelOrder(root) {
  const result = [];
  if (root === null) return result;
  const queue = [root];
  while (queue.length > 0) {
    const levelSize = queue.length;
    const level = [];
    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift();
      level.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(level);
  }
  return result;
}
```

8. Tree DFS

Kab use karein	Problems that require exploring all root-to-leaf paths — path sum, path list, or diameter-style problems where recursion naturally tracks the current path.
Complexity	O(n) time, O(h) space where h is the tree height (recursion stack).
Example problem	Path Sum: Check if the tree has a root-to-leaf path whose values add up to a target sum.

Python

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def has_path_sum(root, target_sum):
    if root is None:
        return False
    if root.left is None and root.right is None:
        return root.val == target_sum
    remaining = target_sum - root.val
    return (has_path_sum(root.left, remaining) or
            has_path_sum(root.right, remaining))
```

Java

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false;
    if (root.left == null && root.right == null) {
        return root.val == targetSum;
    }
    int remaining = targetSum - root.val;
    return hasPathSum(root.left, remaining) || hasPathSum(root.right, remaining);
}
```

C++

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int val) : val(val), left(nullptr), right(nullptr) {}
};

bool hasPathSum(TreeNode* root, int targetSum) {
    if (root == nullptr) return false;
    if (root->left == nullptr && root->right == nullptr) {
        return root->val == targetSum;
    }
    int remaining = targetSum - root->val;
    return hasPathSum(root->left, remaining) || hasPathSum(root->right, remaining);
}
```

JavaScript

```
class TreeNode {
  constructor(val) {
    this.val = val;
    this.left = null;
    this.right = null;
  }
}

function hasPathSum(root, targetSum) {
  if (root === null) return false;
  if (root.left === null && root.right === null) {
    return root.val === targetSum;
  }
  const remaining = targetSum - root.val;
  return hasPathSum(root.left, remaining) || hasPathSum(root.right, remaining);
}
```

9. Subsets / Backtracking

Kab use karein	You need to generate all possible combinations, subsets, permutations, or valid configurations (N-Queens, Sudoku, parentheses).
Complexity	Exponential time in general (O(2^n) for subsets), since it explores the full decision tree — but it prunes invalid paths early.
Example problem	Subsets: Generate all possible subsets of a set of distinct integers.

Python

```python
def find_subsets(nums):
    result = [[]]
    for num in nums:
        result += [subset + [num] for subset in result]
    return result

print(find_subsets([1, 2, 3]))
# [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
```

Java

```java
import java.util.*;

public List<List<Integer>> findSubsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    result.add(new ArrayList<>());
    for (int num : nums) {
        int size = result.size();
        for (int i = 0; i < size; i++) {
            List<Integer> subset = new ArrayList<>(result.get(i));
            subset.add(num);
            result.add(subset);
        }
    }
    return result;
}
```

C++

```cpp
#include <vector>
using namespace std;

vector<vector<int>> findSubsets(vector<int>& nums) {
    vector<vector<int>> result = {{}};
    for (int num : nums) {
        int size = result.size();
        for (int i = 0; i < size; i++) {
            vector<int> subset = result[i];
            subset.push_back(num);
            result.push_back(subset);
        }
    }
    return result;
}
```

JavaScript

```
function findSubsets(nums) {
  let result = [[]];
  for (const num of nums) {
    const size = result.length;
    for (let i = 0; i < size; i++) {
      result.push([...result[i], num]);
    }
  }
  return result;
}
```

10. Modified Binary Search

Kab use karein	The input is sorted (or partially sorted/rotated), and you need better than O(n) search — including 'first/last occurrence' and rotated array problems.
Complexity	O(log n) time, O(1) space.
Example problem	Search in Rotated Sorted Array: Find the index of a target value in a rotated sorted array.

Python

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:      # left half is sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:                            # right half is sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1

print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0))  # 4
```

Java

```
public int searchRotated(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
    return -1;
}
```

C++

```cpp
#include <vector>
using namespace std;

int searchRotated(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
    return -1;
}
```

JavaScript

```javascript
function searchRotated(nums, target) {
  let left = 0, right = nums.length - 1;
  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2);
    if (nums[mid] === target) return mid;
    if (nums[left] <= nums[mid]) {
      if (nums[left] <= target && target < nums[mid]) {
        right = mid - 1;
      } else {
        left = mid + 1;
      }
    } else {
      if (nums[mid] < target && target <= nums[right]) {
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }
  }
  return -1;
}
```

11. Top K Elements (Heap)

Kab use karein	You need the k largest/smallest/most-frequent elements without fully sorting the data.
Complexity	O(n log k) time using a heap of size k, O(k) space — much better than O(n log n) full sort.
Example problem	K Largest Elements: Find the k largest numbers in an array using a min-heap of size k.

Python

```
import heapq

def find_k_largest(nums, k):
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return sorted(min_heap, reverse=True)

print(find_k_largest([3, 1, 5, 12, 2, 11], 3))  # [12, 11, 5]
```

Java

```
import java.util.*;

public List<Integer> findKLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.add(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    List<Integer> result = new ArrayList<>(minHeap);
    result.sort(Collections.reverseOrder());
    return result;
}
```

C++

```cpp
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

vector<int> findKLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int num : nums) {
        minHeap.push(num);
        if ((int)minHeap.size() > k) {
            minHeap.pop();
        }
    }
    vector<int> result;
    while (!minHeap.empty()) {
        result.push_back(minHeap.top());
        minHeap.pop();
    }
    sort(result.rbegin(), result.rend());
    return result;
}
```

JavaScript

```javascript
// JS has no built-in heap; a simple sorted-array approach works for moderate k.
function findKLargest(nums, k) {
  const minHeap = [];
  for (const num of nums) {
    minHeap.push(num);
    minHeap.sort((a, b) => a - b);
    if (minHeap.length > k) {
      minHeap.shift();
    }
  }
  return minHeap.sort((a, b) => b - a);
}
```

12. Dynamic Programming (0/1 Knapsack)

Kab use karein	The problem asks for an optimal value (max/min/count of ways) and has overlapping subproblems + optimal substructure — each item is either included or excluded.
Complexity	O(n * capacity) time and space for the tabulation version; can be optimized to O(capacity) space with a 1-D array.
Example problem	0/1 Knapsack: Given weights and values of n items, find the maximum value that fits in a knapsack of a given capacity.

Python

```
def knapsack(profits, weights, capacity):
    n = len(profits)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for c in range(capacity + 1):
            dp[i][c] = dp[i - 1][c]  # exclude item i-1
            if weights[i - 1] <= c:
                dp[i][c] = max(dp[i][c], dp[i - 1][c - weights[i - 1]] + profits[i - 1])
    return dp[n][capacity]

print(knapsack([1, 6, 10, 16], [1, 2, 3, 5], 7))  # 22
```

Java

```
public int knapsack(int[] profits, int[] weights, int capacity) {
    int n = profits.length;
    int[][] dp = new int[n + 1][capacity + 1];
    for (int i = 1; i <= n; i++) {
        for (int c = 0; c <= capacity; c++) {
            dp[i][c] = dp[i - 1][c];
            if (weights[i - 1] <= c) {
                dp[i][c] = Math.max(dp[i][c], dp[i - 1][c - weights[i - 1]] + profits[i - 1]);
            }
        }
    }
    return dp[n][capacity];
}
```

C++

```
#include <vector>
#include <algorithm>
using namespace std;

int knapsack(vector<int>& profits, vector<int>& weights, int capacity) {
    int n = profits.size();
    vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int c = 0; c <= capacity; c++) {
            dp[i][c] = dp[i - 1][c];
            if (weights[i - 1] <= c) {
                dp[i][c] = max(dp[i][c], dp[i - 1][c - weights[i - 1]] + profits[i - 1]);
            }
        }
    }
    return dp[n][capacity];
}
```

JavaScript

```
function knapsack(profits, weights, capacity) {
  const n = profits.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(capacity + 1).fill(0));
  for (let i = 1; i <= n; i++) {
    for (let c = 0; c <= capacity; c++) {
      dp[i][c] = dp[i - 1][c];
      if (weights[i - 1] <= c) {
        dp[i][c] = Math.max(dp[i][c], dp[i - 1][c - weights[i - 1]] + profits[i - 1]);
      }
    }
  }
  return dp[n][capacity];
}
```

13. Graph BFS / DFS

Kab use karein	Any problem on a graph or grid — connectivity, shortest path in unweighted graphs (BFS), traversal order, or exploring all paths (DFS).
Complexity	O(V + E) time and space for an adjacency-list representation.
Example problem	Graph BFS Traversal: Traverse a graph given as an adjacency list, starting from a source node.

Python

```python
from collections import deque

def bfs(graph, start):
    visited = set([start])
    order = []
    queue = deque([start])
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

graph = {0: [1, 2], 1: [2], 2: [0, 3], 3: [3]}
print(bfs(graph, 2))  # [2, 0, 3, 1]
```

Java

```java
import java.util.*;

public List<Integer> bfs(Map<Integer, List<Integer>> graph, int start) {
    List<Integer> order = new ArrayList<>();
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();
    queue.add(start);
    visited.add(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.add(neighbor);
            }
        }
    }
    return order;
}
```

C++

```cpp
#include <vector>
#include <queue>
#include <unordered_set>
#include <unordered_map>
using namespace std;

vector<int> bfs(unordered_map<int, vector<int>>& graph, int start) {
    vector<int> order;
    unordered_set<int> visited = {start};
    queue<int> q;
    q.push(start);
    while (!q.empty()) {
        int node = q.front(); q.pop();
        order.push_back(node);
        for (int neighbor : graph[node]) {
            if (visited.find(neighbor) == visited.end()) {
                visited.insert(neighbor);
                q.push(neighbor);
            }
        }
    }
    return order;
}
```

JavaScript

```javascript
function bfs(graph, start) {
  const order = [];
  const visited = new Set([start]);
  const queue = [start];
  while (queue.length > 0) {
    const node = queue.shift();
    order.push(node);
    for (const neighbor of graph[node] || []) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }
  return order;
}
```

14. Monotonic Stack

Kab use karein	You need the 'next greater/smaller element' for every element in an array, or to solve problems about visibility, spans, or histogram areas.
Complexity	O(n) time even though it looks like nested loops — each element is pushed and popped from the stack at most once.
Example problem	Next Greater Element: For each element, find the next element to its right that is greater than it.

Python

```
def next_greater_elements(nums):
    result = [-1] * len(nums)
    stack = []  # stores indices
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
            result[stack.pop()] = num
        stack.append(i)
    return result

print(next_greater_elements([2, 1, 3, 4, 1]))  # [3, 3, 4, -1, -1]
```

Java

```
import java.util.*;

public int[] nextGreaterElements(int[] nums) {
    int[] result = new int[nums.length];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

C++

```cpp
#include <vector>
#include <stack>
using namespace std;

vector<int> nextGreaterElements(vector<int>& nums) {
    vector<int> result(nums.size(), -1);
    stack<int> st; // stores indices
    for (int i = 0; i < (int)nums.size(); i++) {
        while (!st.empty() && nums[st.top()] < nums[i]) {
            result[st.top()] = nums[i];
            st.pop();
        }
        st.push(i);
    }
    return result;
}
```

JavaScript

```javascript
function nextGreaterElements(nums) {
  const result = new Array(nums.length).fill(-1);
  const stack = []; // stores indices
  for (let i = 0; i < nums.length; i++) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] < nums[i]) {
      result[stack.pop()] = nums[i];
    }
    stack.push(i);
  }
  return result;
}
```

15. Union-Find (Disjoint Set)

Kab use karein	Problems about grouping, connectivity, or detecting cycles in an undirected graph — 'are these two nodes connected', 'how many groups exist'.
Complexity	Nearly O(1) per operation (O(α(n)) amortized) when using path compression and union by rank.
Example problem	Number of Connected Components: Given n nodes and a list of edges, find the number of connected components.

Python

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        root_x, root_y = self.find(x), self.find(y)
        if root_x == root_y:
            return
        if self.rank[root_x] < self.rank[root_y]:
            root_x, root_y = root_y, root_x
        self.parent[root_y] = root_x
        if self.rank[root_x] == self.rank[root_y]:
            self.rank[root_x] += 1
        self.count -= 1

def count_components(n, edges):
    uf = UnionFind(n)
    for a, b in edges:
        uf.union(a, b)
    return uf.count
```

Java

```
class UnionFind {
    int[] parent, rank;
    int count;

    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    void union(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return;
        if (rank[rootX] < rank[rootY]) { int t = rootX; rootX = rootY; rootY = t; }
        parent[rootY] = rootX;
        if (rank[rootX] == rank[rootY]) rank[rootX]++;
        count--;
    }
}

public int countComponents(int n, int[][] edges) {
    UnionFind uf = new UnionFind(n);
    for (int[] edge : edges) uf.union(edge[0], edge[1]);
    return uf.count;
}
```

C++

```cpp
#include <vector>
using namespace std;

class UnionFind {
public:
    vector<int> parent, rank_;
    int count;

    UnionFind(int n) : parent(n), rank_(n, 0), count(n) {
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    void unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return;
        if (rank_[rootX] < rank_[rootY]) swap(rootX, rootY);
        parent[rootY] = rootX;
        if (rank_[rootX] == rank_[rootY]) rank_[rootX]++;
        count--;
    }
};

int countComponents(int n, vector<vector<int>>& edges) {
    UnionFind uf(n);
    for (auto& edge : edges) uf.unite(edge[0], edge[1]);
    return uf.count;
}
```

JavaScript

```
class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);
    this.rank = new Array(n).fill(0);
    this.count = n;
  }

  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]);
    }
    return this.parent[x];
  }

  union(x, y) {
    let rootX = this.find(x), rootY = this.find(y);
    if (rootX === rootY) return;
    if (this.rank[rootX] < this.rank[rootY]) [rootX, rootY] = [rootY, rootX];
    this.parent[rootY] = rootX;
    if (this.rank[rootX] === this.rank[rootY]) this.rank[rootX]++;
    this.count--;
  }
}

function countComponents(n, edges) {
  const uf = new UnionFind(n);
  for (const [a, b] of edges) uf.union(a, b);
  return uf.count;
}
```